UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Nombre: Abigail Quinga NRC: 36460
Fecha:
Materia: Fundamentos de programación
Carrera: Electrónica y Automatización

Problema 1.1 Media de dos valores
Desarrolla un programa que ingrese dos números por teclado e imprima su media.
Realiza y completa la tabla de objetos del problema 1.1:

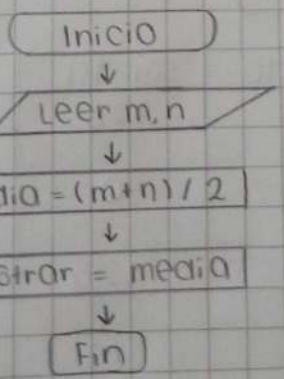
1. Análisis del problema
   Entradas: Dos números enteros (m, n)
   Proceso: Sumar m y n, después dividir por 2
   Salida: La media de m y n
   Restricción: m y n deben ser números enteros (positivos, cero o negativos).
2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Entrada (m)	m	var	entero
Entrada (n)	m	var	entero
Salida	media	Promedio	entero

3. Seudocódigo

```
Inicio
    Leer m, n, media entero;
    Escribir "Ingrese el primer número" ;
    Leer m;
    Escribir "Ingrese el segundo número";
    Leer n;
    media <- (m + n) / 2
    Escribir "La media es: ", media;
Final
```

4. Diagrama de flujo



5. Prueba de escritorio

m	n	media	salida
50	-	-	-
50	70	-	-
50	70	60	-
50	70	60	media: 60

Problema 1.2 valor absoluto de x al cubo
Desarrolla un programa que lea un número por teclado y escriba por pantalla $|x|^3$.

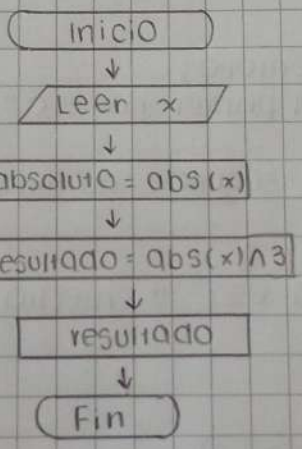
1. Análisis del problema
   Entrada: Un número x real
   Proceso: Calcular el valor absoluto de x y elevarlo al cubo
   Salida: valor de $|x|^3$
   Restricción: Ingresar un dato real puede ser positivo, negativo o el cero.
2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Entrada	x	var.	real
Valor absolu.	| |	var.	real
Salida	resultado	$\|x\|^3$	real

3. Seudocódigo

```
Inicio
    Leer x, absoluto, resultado;
    Escribir "Ingrese el valor de x:";
    Leer x
    absoluto <— abs(x)
    resultado <— abs(x) ^ 3
    Escribir "Resultado es:", resultado
Final
```

4. Diagrama de flujo



5. Prueba escritorio

x	abs(x)	resultado	salida
-5	|-5|	5^3	125
0	|0|	0^3	0
5	|5|	5^3	125

Problema 1.3 Divisible

Desarrolla un programa que lea dos números enteros por teclado y determine si el primero es divisible por el segundo. Se mostrará por pantalla el resultado

Pista: La solución utiliza el operador módulo (%), que devuelve el resto de la división entera entre números enteros. Si x es divisible entre y, el resto debe ser 0.

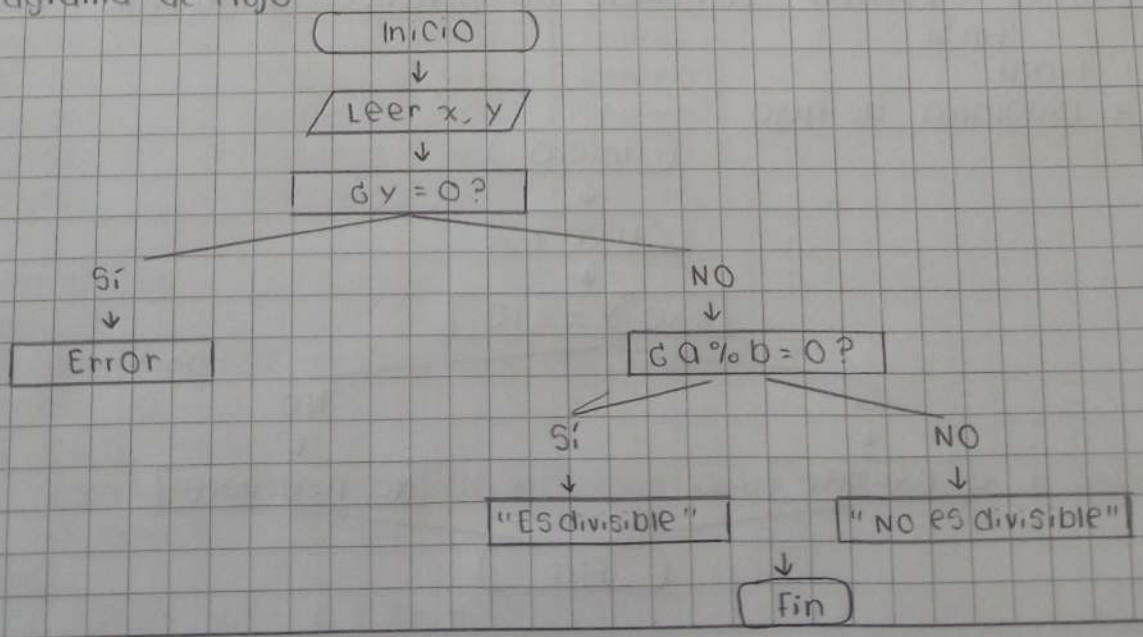
1. Análisis del problema
   Entrada: Dos números enteros (x, y)
   Proceso: Calcular si x es divisible para y mediante el operador módulo (x % y) cuando el resto es igual a cero
   Salida: Mensaje indicando si x es o no divisible por y
   Restricción: y no puede ser 0, la división entre cero no existe
2. Tabla de objetos

Objeto	Nombre	valor	Tipo
Entrada	x	var.	entero
Entrada	y	var.	entero
cero	0	cte	entero

3. Seudocódigo

```
Inicio
    Leer x, y
    Escribir "Ingrese el valor de x:";
    Leer x
    Escribir "Ingrese el valor de y:";
    Leer y
    Si y = 0 entonces
        Escribir "No es posible dividir entre cero"
    Sino
        Si x mod y = 0 entonces
            Escribir x, " es divisible por", y
        Sino
            Escribir x, " no es divisible por", y
        Fin Si
    Fin Si
Fin
```

4. Diagrama de flujo



5. Prueba de escritorio

x	y	¿y=0?	x mod y	Salida
12	3	NO	0	12 es divisible para 3
8	2	NO	0	8 es divisible para 2
5	0	Si	-	No es posible dividir entre 0

Problema 1.4 intervalo
Desarrolla un programa que lea un número por teclado y determine si pertenece al intervalo [0, 10], indicando el resultado por pantalla
Pista: El número introducido debe cumplir $x \geq 0$ y $x \leq 10$ (ambas condiciones a la vez) para pertenecer al intervalo. Para establecer estas 2 condiciones se utiliza un operador lógico: el operador y en el diagrama de flujo y && en C. Evita la tentación de escribir $0 \leq x \leq 10$ como una sola condición, porque en C no se evalúa como esperas.

1. Análisis del problema
   Entrada: Un número x
   Proceso: Evaluar si pertenece $x \geq 0$ y $x \leq 10$
   Salida: Mensaje si pertenece o no al intervalo [0; 10]
   Restricción: En código C o diagramación utilizar un operador lógico (&& / y) no en $0 \leq x \leq 10$

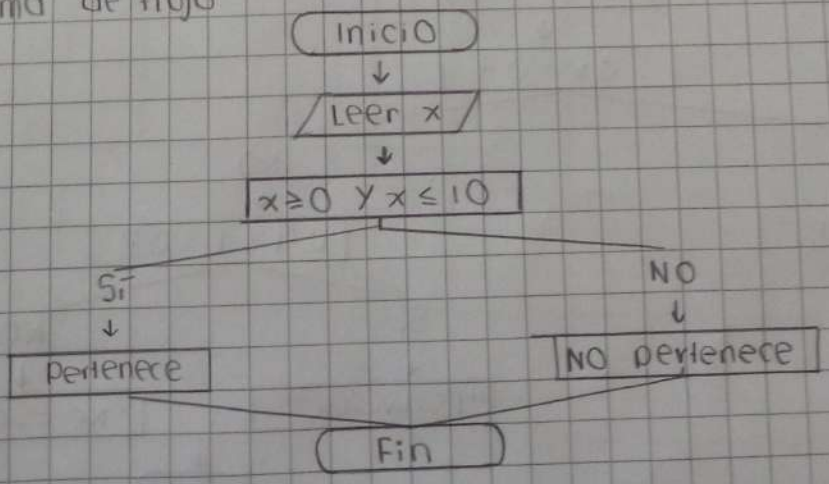
2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Entrada	x	var.	Real
condición	pertenece	-	Lógico

3. Seudocódigo

```
Inicio
    Escribir "Ingrese un número:";
    Leer x
    Si x >= 0 y x <= 10, entonces;
        Escribir "El número pertenece al intervalo";
    Sino
        Escribir "El número no pertenece al intervalo";
    Fin Si
Final
```

4. Diagrama de flujo



5. Prueba de escritorio

x	¿x≥0?	¿x≤10?	Salida
9	Si	Si	Pertenece al intervalo
10	Si	Si	Pertenece al intervalo
-2	NO	NO	No pertenece al intervalo

Problema 1.5 Conversión de unidades de tiempo

Desarrolla un programa que lea por teclado un valor entero c, interpretandolo como el número de segundos que duró un evento y calcule y muestre por pantalla cuántos días, horas, minutos y segundos representa

Para calcular los días, horas, minutos y segundos se utilizan la división entera y el módulo (resto de una división, operador %)

Sea c el número de segundos a convertir. como un día tiene 24 horas, una hora tiene 60 minutos y un minuto tiene 60 segundos el cociente entero c/(24·60·60) es el número de días completos contenidos en c. Asimismo, c%(24·60·60) es el número de segundos restantes tras descontar esos días. se procede de igual manera para obtener las horas (1 hora = 60 60 segundos) y los minutos (1 minuto = 60 segundos)

Ejemplo: supón que quiere convertir c= 10000 segundos. Como 10 000 / (24·60·60) = 0, hay 0 días. porque 10000 segundos es menos que un día. Análogamente, 10 000 / 60·60 = 2 horas. Si a 10 000 segundos les quitas esas 2 horas quedan 10000 % 60·60 = 2 800 segundos, que son algo más de 46 minutos, cantidad que se obtiene con 2 800 / 60 = 46 minutos Finalmente 2 800 % 60 = 40 segundos. Por lo tanto, 10 000 segundos equivalen a 0 días, 2 horas, 46 minutos y 40 segundos.

1. Análisis del problema

Entradas: Valor entero c, representa el número de segundos

Procesa: Calcular cuántos días, horas, minutos y segundos utilizando el operador módulo.

Salida: Tiempo obtenido

Restricción: NO debe de ser un número c < 0, solo c ≥ 0 para cumplimiento de un número válido.

2. Tabla de Objetos

Objetos	Nombre	Valor	Tipo
Entrada	c	var.	Entero
Salida	días	var.	Entero
Salida	horas	var.	Entero
Salida	minutos	var.	Entero
Salida	segundos	var.	Entero

3. Seudocódigo

Inicio

Escribir "Ingrese la cantidad de segundos:";

Leer c

Si c < 0, entonces;

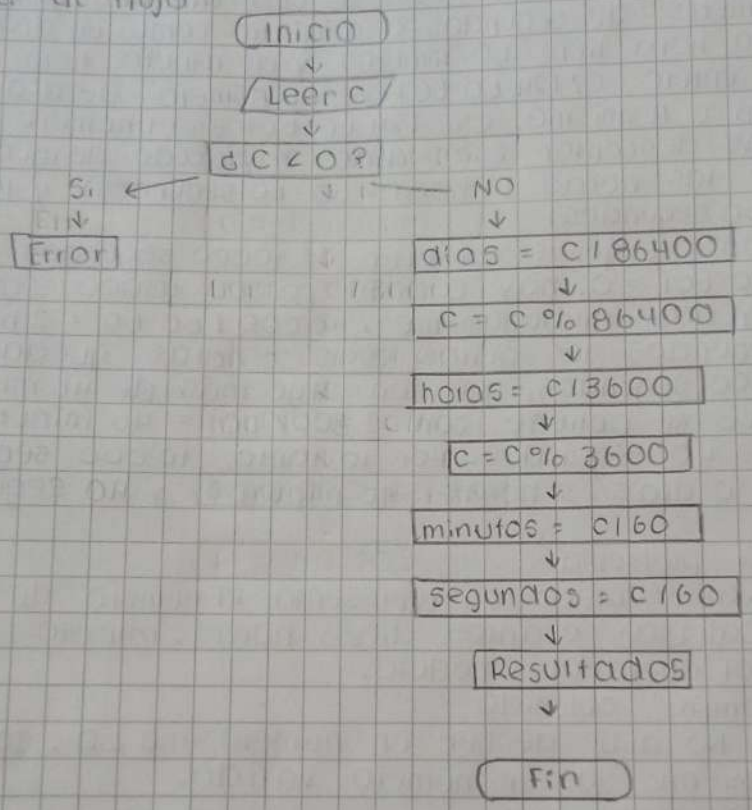
Escribir "El número de segundos debe ser mayor o igual a cero"

Sino

días ← c / 86,400

```
resto <- c % 86,400
horas <- resto / 3600
resto <- resto % 3600
minutos <- resto / 60
segundos <- resto / 60
Escribir "Dias:", dias;
Escribir "Horas:", horas;
Escribir "Minutos:", minutos;
Escribir "segundos:", segundos;
    Fin Si
Fin
```

4. Diagrama de flujo



5. Prueba de escritorio

c	días	resto	horas	minu.	segun.	Salidas
10000	-	-				
10000	0	10000				
10000	0	2800	2			
10000	0	40	2	46		
10000	0	40	2	46	40	Resultado

Problema 1.6 Máximo, mínimo y media de alturas

Desarrolla un programa que lea por teclado un número indeterminado de alturas y deje de leer cuando se introduzca una negativa. El programa debe calcular e imprimir la media el máximo y el mínimo de las alturas introducidas

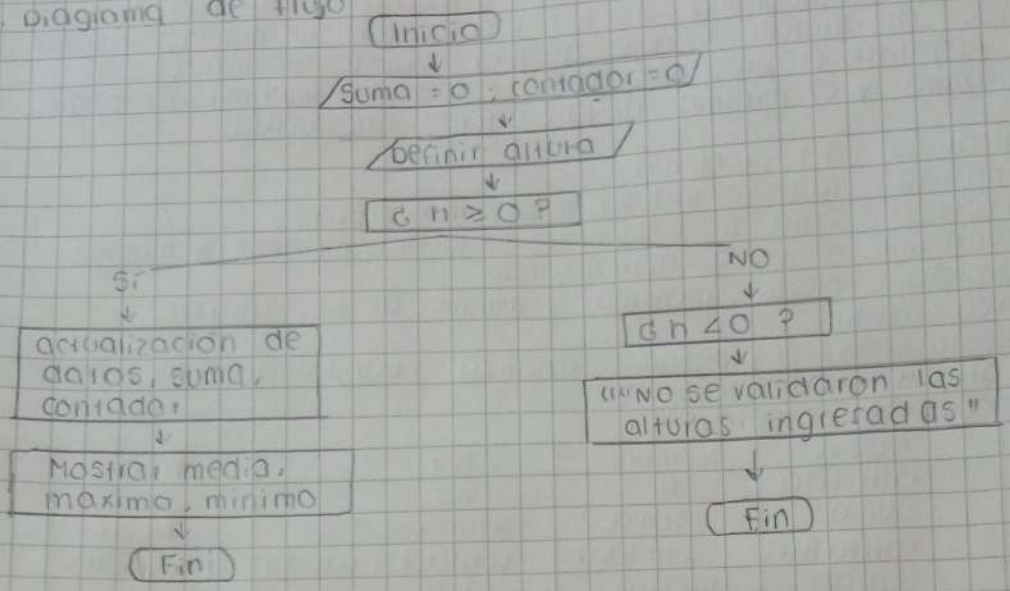
1. Análisis del problema
   Entrada: Número indeterminado de alturas
   Proceso: Calcular e imprimir la media, máximo y mínimo conforme se vaya actualizando.
   Salida: Media, máximo y mínimo de las alturas registradas
   Restricción: El programa finaliza cuando detecta una altura $h < 0$.
2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Entrada	h	var.	Real
Suma	suma	suma válida	Real
Cantidad	contador	contador válida	Entero
Salida	media	var.	Real
Salida	máximo	var.	Real
Salida	mínimo	var.	Real

3. Seudocódigo

```
Inicio
    Definir altura, suma, máximo, media, mínimo como Real
    Definir contador como Entero
    suma = 0
    cont = 0
    Escribir "Ingrese una altura negativa para finalizar:";
    Leer h
    Si h >= 0, entonces ;
        máximo <- h
        mínimo <- h
    Sino h >= 0; entonces ;
        suma <- suma + h
        cont <- cont + 1
    Fin Si
    Si contador > 0, entonces ;
        media <- suma % contador
        Escribir "Media";
        Escribir "Mínimo";
        Escribir "Máximo";
    Sino
        Escribir "NO se validaron las alturas ingresadas"
    Fin Si
Final
```

4. Diagrama de flujo



5. Prueba de escritorio

h	suma	cont	máx	min	med	salida
2	0	0	2	2		
1,50	1,50	1	2	1,50		
1,70	3,20	2	1,70	1,50		
-1,20	3,20	2	1,70	1,50	1,60	media: 1,60
						máx: 1,70
						minimo: 1,50

Problema 1.7 Raiz cuadrada entera

Desarrolla un programa que calcule la raíz cuadrada entera por defecto de un número N positivo dado. que se define como el mayor número entero $R > 0$ tal que $R^2 \leq N$.

Pista: Dada una variable entera $i = 1$, itera incrementando su valor de uno mientras $i^2 \leq N$. La aproximación por defecto de la raiz es $R = i - 1$.

1. Análisis del problema

   Entrada: Número N positivo

   Proceso: incrementar su valor $i^2 \leq N$

   Salida: $R = i - 1$

   Restriccion: N debe ser mayor que 0.

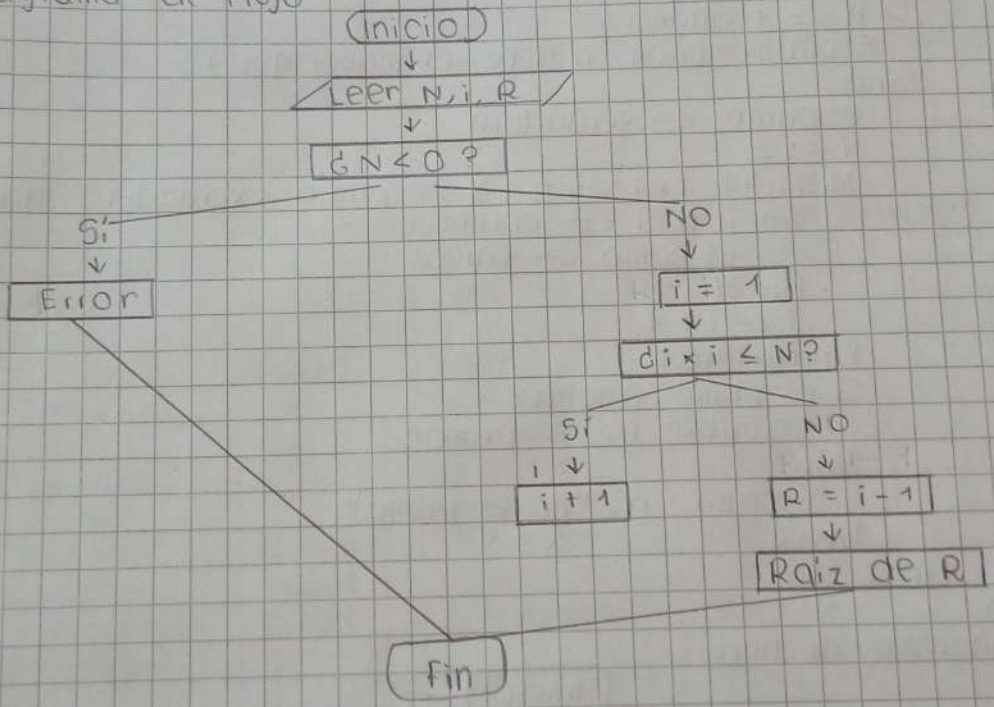
2. Tabla de objetos

Objeto	Nombre	valor	Tipo
Número positivo	N	var.	Entero
variable	i	var.	Entero
Raiz cuadrada Entera	R	var.	Entero

3. Seudocodigo

```
Inicio
    Leer N, i, R como Entero
    Escribir "Ingrese un entero positivo"
    Leer N
    Si N <= 0, entonces
        Escribir "El número debe ser entero positivo mayor
        que cero"
    Sino
        i <- 1.
        Mientras ixi<= N Hacer
            i <- i + 1
        Fin Mientras
        R <- i - 1
        Escribir "La raiz cuadrada entera de N es:", R
    Fin Si
Fin.
```

4. Diagrama de flujo



5. Prueba de escritorio

N	i	ixi	condición	R	Salida
15	1	1	1		
15	2	2	1		
15	3	9	1	2	Raiz entera de 15 es 3
15	4	16	0	3	

Problema 1.8 Número primo

Desarrolla un programa que ingrese por teclado un número entero mayor que 1 y determine si es primo o no.

1. Análisis del problema

   Entrada: n número entero mayor que 1

   Proceso: calcular si existe algún divisor desde 2

   Salida: "es primo" o "no es primo"

   Restricción: n > 1

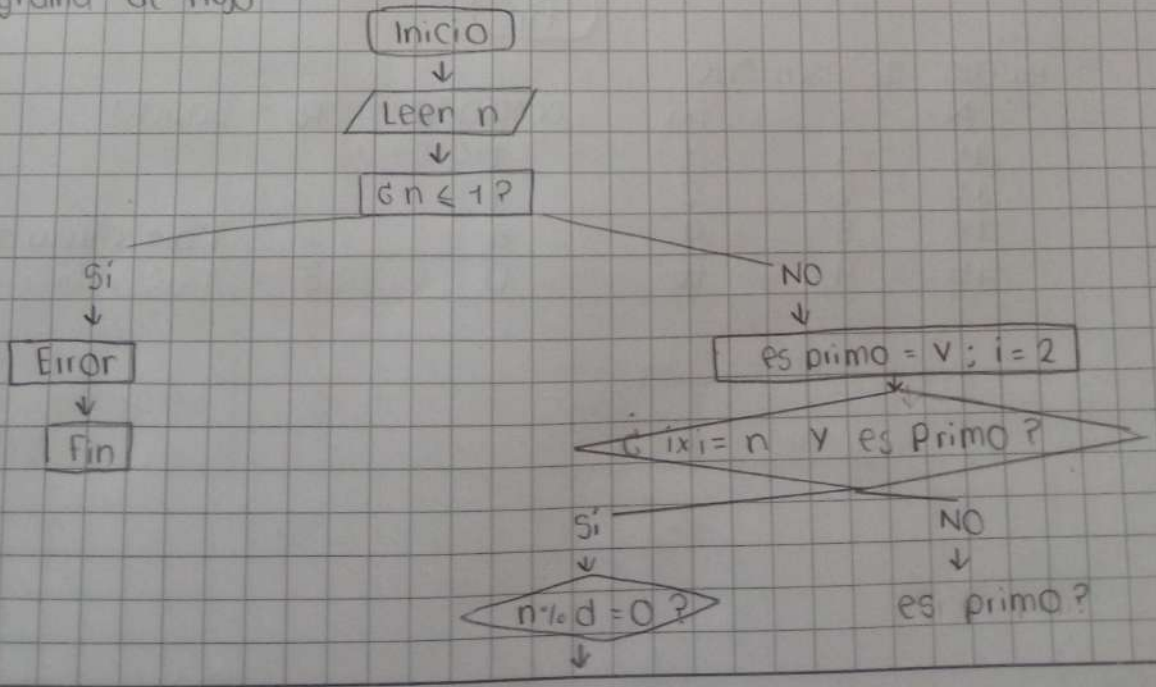
2. Tabla de objetos

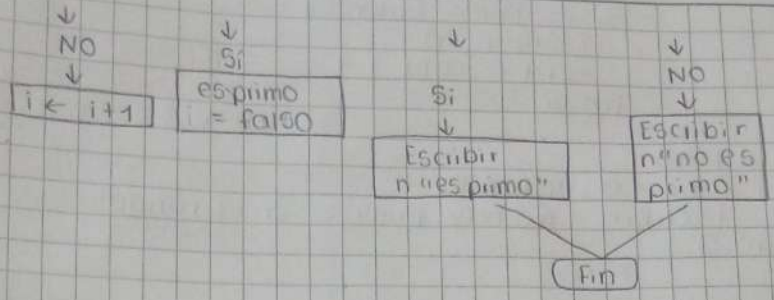
Objetos	Nombre	Valor	Tipo
Entrada	n	n > 1	Entero
Divisor	i	2	Entero
Salida	es primo	1	Lógico

3. Seudocódigo

```
Inicio
    Leer n
    Si n <= 1 entonces
        Escribir "Error: n debe ser mayor que 1"
    Sino
        es primo <- verdadero
        i <- 2
        Mientras i x i <= n Y es primo = verdadero Hacer
            Sin n % i = 0 entonces
                es primo <- falso
            Fin Si
            i <- i + 1
        Fin Mientras
        Si es primo entonces
            Escribir n, "es primo"
        Sino
            Escribir n, "no es primo"
        Fin Si
    Fin Si
Fin
```

4. Diagrama de flujo





5. Prueba de escritorio

n	i	Prueba	es primo	salida
7	2	$2 \cdot 2 = 4 > 2$	V	"es primo"
9	2	$4 \leq 9$; $9 \% 2 = 1$	F	"NO es primo"
		$i = 3 \rightarrow 9 \% 3 = 0$		

Problema 9. Números primos comprendidos entre dos números dados

Desarrolla un programa que muestre por pantalla todos los números primos comprendidos entre dos números enteros introducidos por teclado.

1. Análisis del problema
   Entrada: Dos enteros (m y p)
   Proceso: Recorrer los números comprendidos entre ambos extremos
   Salida: comprobar todos los primos en ese rango
   Restricción: m y p mayores que 1
2. Tabla de objetos

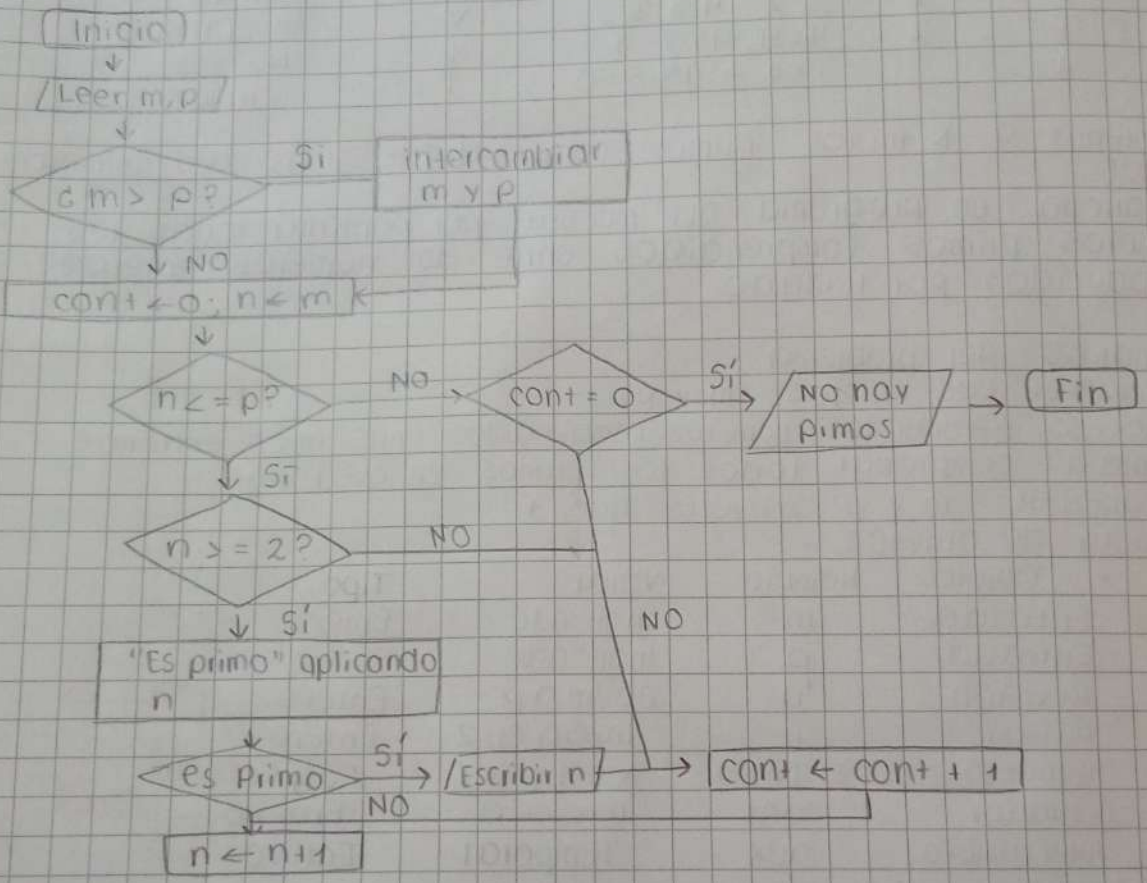
Objeto	Nombre	valor	Tipo
Entrada	m	ingresado	Entero
Entrada	p	Ingresado	Entero
Revisión	n	De m a p	Entero
Divisor	i	Inicia en 2	Entero
Bandera	es Primo	V o F por n	Lógico
Contador	cont	Inicia 0	Entero
Intercambio	aux	Temporal	Entero

3. Seudocódigo

```
Inicio
   Leer m, p
   Si m > p, entonces
     aux <- m
     m <- p
     b <- aux
   Fin Si
   cont <- 0
   Para n <- m hasta b Hacer
     Si n >= 2 entonces
        es Primo <- verdadero
        i <- 2
        Mientras ixi <= n Y es Primo Hacer
           Si n % i = 0, entonces es Primo <- Falso
           i <- i + 1
        Fin Mientras
```

```
        Si es primo entonces
            Escribir n
            cont ← cont + 1
        Fin Si
    Fin Si
Fin Para
Si cont = 0, entonces
    Escribir "No hay primos en el rango"
Fin Si
Fin
```

4. Diagrama



5. Prueba de escritorio

m	p	Revisión	Salida
12	16	13 ✓ 15 ✓	13, 15
16	12	13 ✓ 15 ✓	13, 15

Problema 10. Factorial

Desarrolla un programa que lea un número entero $n \geq 0$ y calcule $n! = 1 \cdot 2 \cdot \ldots \cdot n$, sabiendo que $0! = 1$

Ejemplo $n = 5 \rightarrow 120$

Pista: Usa un acumulador que inicie en 1 y se multiplique en cada vuelta del ciclo

Para pensar: ¿Qué ocurre si el usuario ingresa un número negativo? ¿y con $n = 20$ en una variable de tipo int

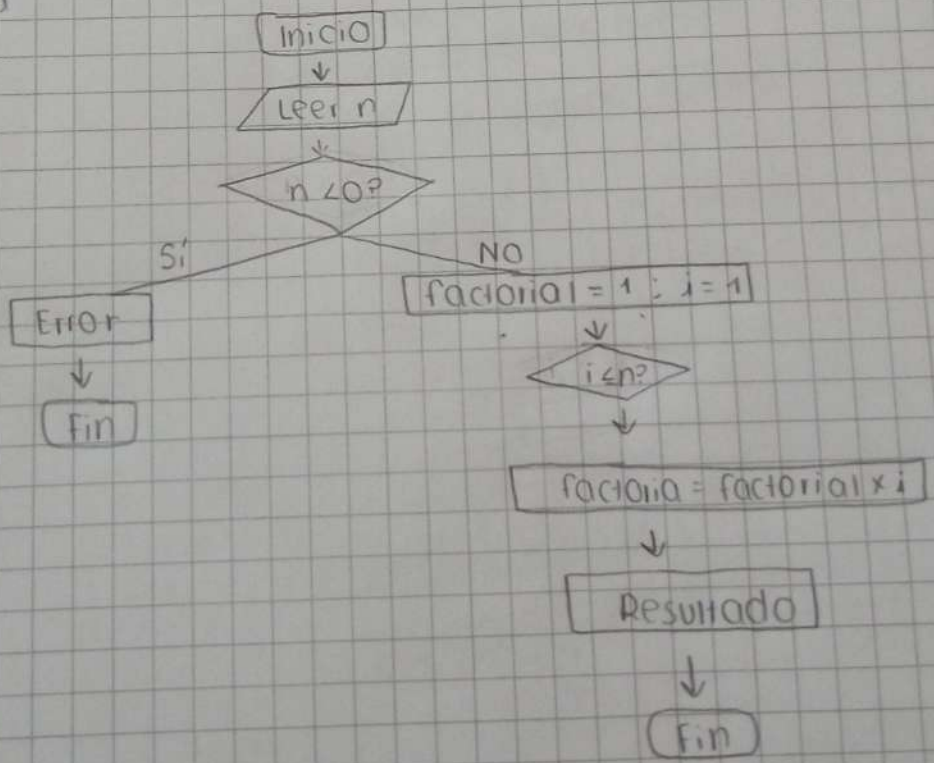
1. Análisis de datos
   Entrada: n entero ≥ 0
   Proceso: multiplicar n con un acumulador inicial 1.
   Con 0! = 1 queda el valor inicial
   Salida: n!
   Restricción: n no valor negativo
2. Tabla de objetos

Objeto	Nombre	valor	Tipo
número	n	n ≥ 0	Entero
contador	i	De 2 a n	Entero
acumulador	f	Inicia 1	Entero largo

3. Seudocódigo

```
Inicio
    Leer n
    si n <0, entonces
      Escribir "Error : el factorial no existe número negativo"
    sino
      f <- 1
      para i <- 2 hasta n hacer
         f <- f x i
      Fin para
      Escribir n, "! = ", f
    Fin Si
Fin
```

4. Diagrama



5. Prueba

n	factorial	Salida
5	1x2x3x4x5	120
1	1	1
-5		Error

Para pensar

→ Como el factorial no está definido, el programa arroja un resultado erroneo, por lo cual es importante validar

→ int : 20! = 2432902008176640000, este valor supera el límite, int de 32 bit se admite un máximo valor de 2147483647. Por ende para almacenan un 20! se necesita un tipo entero mayor capacidad